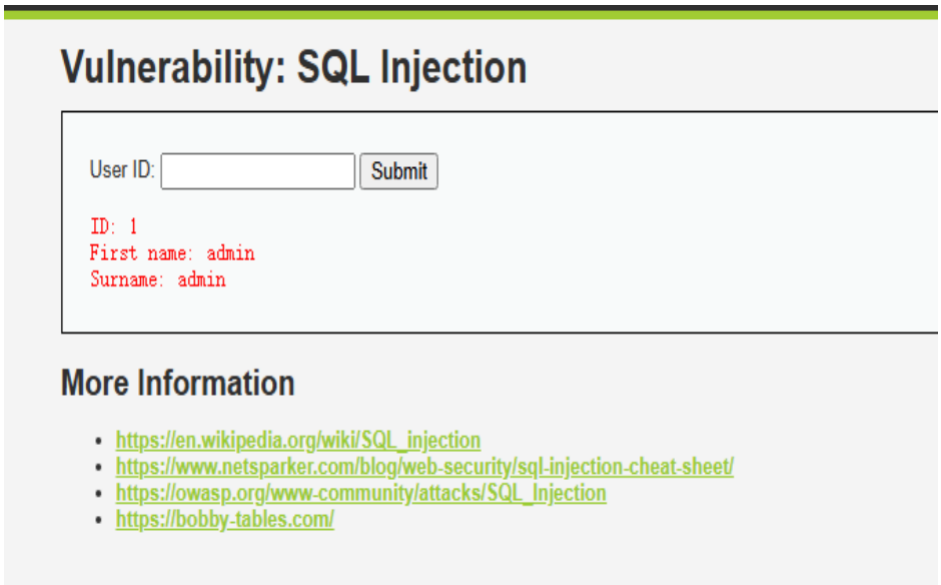
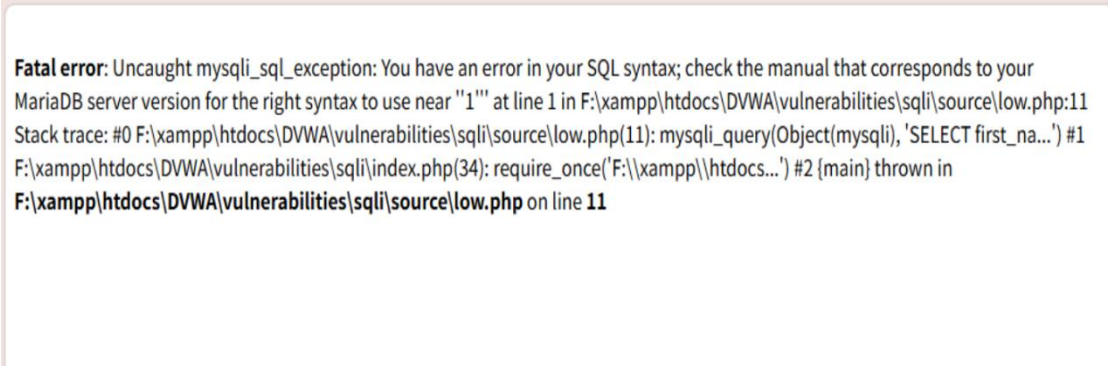
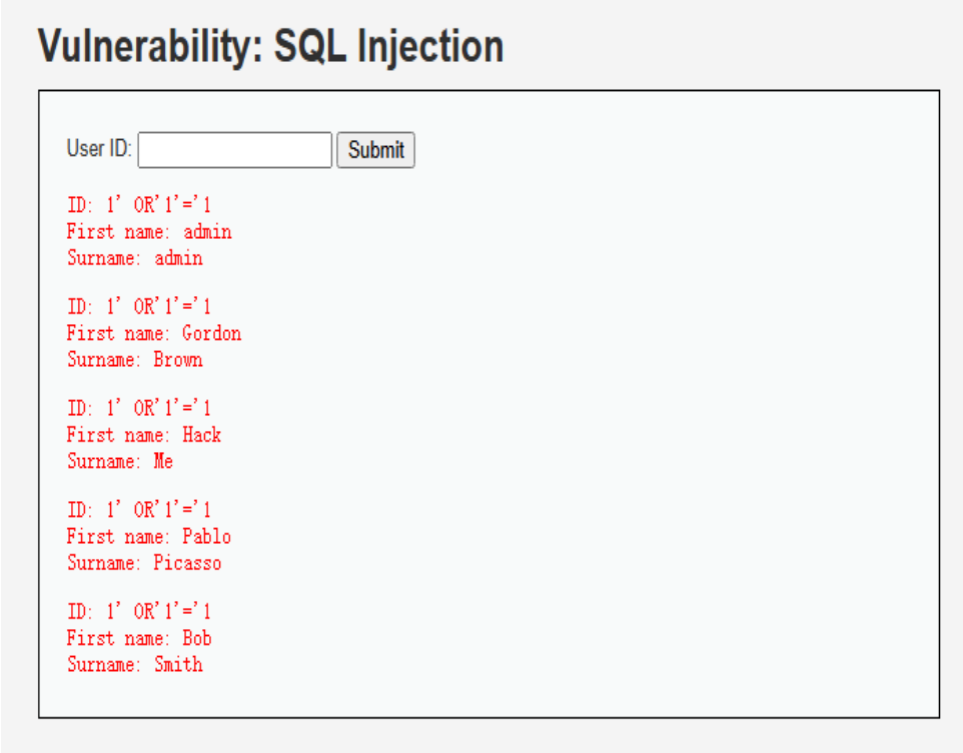
业务场景：假设是一个“用户 ID 查询”功能。
最小业务链路：用户在网页输入框输入 1 → 后端代码接收 1 → 拼接到 SQL 语句 SELECT * FROM users WHERE id = 1 → 数据库执行 → 网页显示用户信息。
为什么有关：因为网页的数据（如用户信息）存储在数据库中，页面输入是查询数据的“钥匙”。
如何进入查询：通过代码中的字符串拼接（例如 PHP 的 $query = "SELECT * FROM users WHERE id = $id";）。
创造可能性的原因：因为代码盲目信任了用户的输入，将用户输入的“数据”直接当成了“代码逻辑”的一部分去执行。



正常查询 ID 为 1 的用户，页面返回了预期的用户 First Name 和 Surname。



输入单引号破坏了原有的 SQL 闭合结构，导致数据库解析失败并报错。这说明后端没有对特殊字符进行过滤。



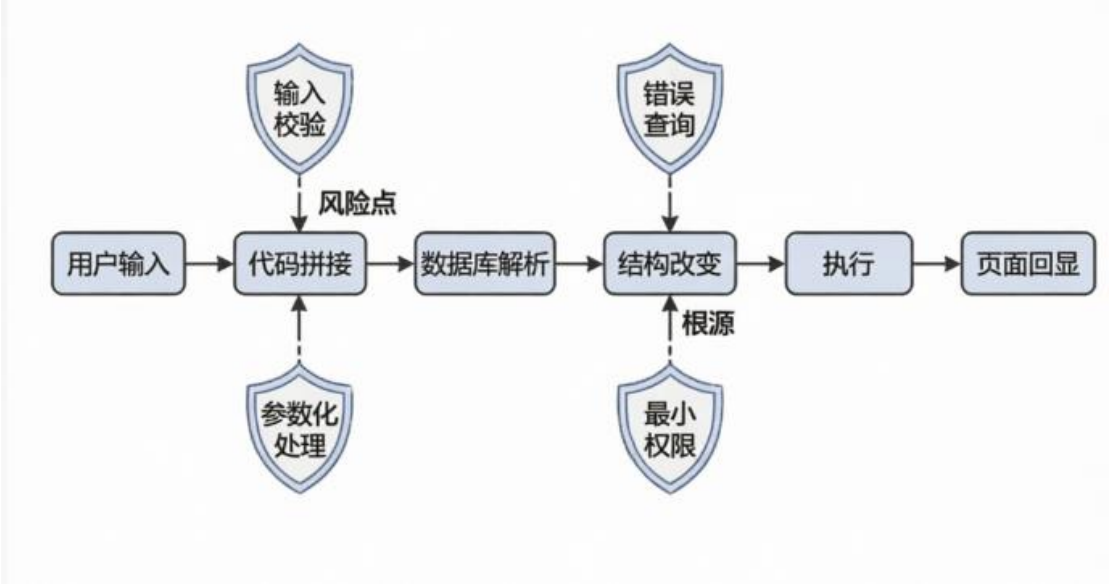
输入改变了原本的查询逻辑，OR '1'='1 使得条件永远为真，导致数据库返回了所有记录

输入如何进入结构：
描述：用户输入的数据未经过滤，直接通过字符串连接符（如 PHP 中的 .）嵌入到 SQL 语句字符串中。
结构被污染 vs 字符危险：
核心论点：单引号本身只是普通字符，只有当它被拼接到 SQL 语句中，被数据库解析器当作“字符串结束符”处理时，才具有破坏力。如果使用了预编译，单引号只会被视为普通文本，不会破坏结构。
Low 与 Medium 差异：
去哪里：切换 DVWA 安全级别到 Medium。
操作：尝试输入 1' OR '1'='1，发现失败。尝试输入 1 OR 1=1（数字型注入），发现成功。
分析：Medium 级别使用了 mysql_real_escape_string 转义了单引号，导致字符型注入失效，但未对数字型输入做强制类型转换，留下了数字型注入的缺口。
普通注入 vs 盲注 (Blind)：
差异：普通注入页面直接回显数据库内容或报错信息；盲注页面不显示数据，只显示“对/错”或“有/无”的页面差异。
证据形式：普通注入看数据；盲注看响应时间（Time-based）或页面布尔值变化（Boolean-based）。

SQL 注入的根源更接近？
答案：B. 输入进入了不该进入的查询结构
理由：任何字符在数据区都是安全的，只有当它突破了数据区的边界，进入了代码执行区，

改变了 SQL 语法树的结构，才构成注入。
Blind SQL Injection 和普通 SQL 注入的最大差异更接近？
答案：B. 证据形式不同
理由：两者的成因完全一致（都是代码与数据未分离）。区别仅在于应用程序的反馈机制：普通注入直接回显数据，盲注只能通过逻辑判断（真/假）或时间延迟来推断数据。

参数化查询（预编译）
解决什么：彻底解决结构被污染问题。
控制哪一层：查询构造层（根源控制）。
不能替代：不能替代输入校验（如数据类型检查）。
输入校验（白名单）
解决什么：确保数据符合预期格式（如 ID 必须是数字）。
控制哪一层：输入层（暴露面收敛）。
不能替代：不能替代参数化查询（特殊场景下仍需转义）。
错误处理
解决什么：防止数据库报错信息泄露（如表名、字段名）。
控制哪一层：回显层（后果范围控制）。
不能替代：无法阻止注入发生，只能让攻击者“瞎”掉。
最小权限
解决什么：限制数据库账号权限（如禁止 DROP TABLE）。
控制哪一层：执行层（后果范围控制）。
收口 1：



收口 2
我本章最清楚的一点是什么？
我本章最清楚的是 SQL 注入的基本原理，即攻击者如何通过构造恶意输入来改变原有 SQL 查询语句的逻辑，从而获取非授权数据或执行未授权操作。通过 DVWA 平台的实践，我能

够清晰地理解从输入点到结果输出的整个攻击链路。
我最容易混淆的一点是什么？
我最容易混淆的是不同 SQL 注入类型（如联合查询注入、布尔盲注、时间盲注）之间的具体应用场景和判断依据。尤其是在面对没有明显错误回显的注入点时，如何准确区分是布尔盲注还是时间盲注，有时会感到困惑。
我本次作业中哪一段最需要教师复核？
我本次作业中关于“根因判断”和“防护控制点”的部分最需要教师复核。因为我对于如何精准定位到代码层面的根本原因，以及提出的防护措施是否全面、有效，存在一些不确定，希望老师能给予指导。
如果重做一次，我会优先改哪一部分？
如果重做一次，我会优先改进“结果差异或行为证据”部分。我会尝试收集更丰富、更具说服力的证据（例如，更详细的数据库响应截图、更精确的注入语句执行效果对比），来更有力地支撑我的分析结论。
收口 3
1.本作业哪些内容使用过 AI 辅助？
画关系图的思路： 我卡壳的时候问了 AI，让它帮我梳理一下 SQL 注入的因果链条，比如“输入点”之后接什么环节比较顺畅，AI 给了我一个逻辑框架参考。
专业术语解释： 在做 DVWA 实验时，我对“布尔盲注”和“时间盲注”的区别有点晕，问了 AI 让它用通俗的话解释一下，帮我理清了概念。
自评表的润色： 写自评表时，我大概写了几个点，让 AI 帮我把语句组织得通顺一点，看起来更像书面语。
2. 哪些判断是你自己复核后保留的？
DVWA 实验的具体操作步骤： 比如 Burp Suite 怎么设置代理、怎么拦截包、具体的 Payload 怎么写（比如' or '1'='1），这些都是我自己一步步试出来的，AI 只是大概说了原理，具体操作还得靠自己。
截图和证据： 所有的实验截图（比如成功注入的页面、数据库版本号显示）都是我自己在虚拟机里跑出来的，这部分没法造假，必须自己保留。
根因分析的核心观点： 虽然 AI 提了一些代码层面的原因，但我最终保留的是结合我们课程里讲过的“未过滤特殊字符”这个核心点，因为我觉得这最贴合老师上课讲的内容。
3. 哪些内容你发现 AI 输出不准确并进行了修正？
关于防护措施的细节： AI 一开始建议我用“防火墙”这种比较笼统的说法，我觉得太虚了，就改成了具体的“预编译语句（Prepared Statements）”和“输入过滤”，因为这才是我们课本上强调的。
工具版本的描述： AI 提到的 Burp Suite 界面有些按钮位置和我电脑上装的不一样（版本差异），我按照我实际操作的界面描述进行了修正，避免误导。
4. 你最终如何确认结论和证据是一一对应的？
自己重跑了一遍实验： 写完报告后，我重新在 DVWA 上操作了一遍，确保我写的每一个结论（比如“存在注入漏洞”）都能对应上我截的那张图（比如“页面显示了数据库版本”）。
对照课本检查： 我把我的“根因判断”和课本上的定义比对了一下，确认逻辑是通的，不是 AI 瞎编的幻觉。